Week 8. Problem set (solutions by Karim Khabibrakhmanov)

1. Perform Heap-Sort [CLRS, §6.4] on the following input array:

2	3	0	4	7	1	5	8	6

Show the state of the array after each call to Max-Heapify from the Heap-Sort procedure (solution must have 8 arrays).

Answer:
Initially, we need to apply Max-Heapify to our array. Here is our resulting array:

8	7	5	6	2	1	0	3	4

Next, for all other arrays, we apply an algorithm that will swap the first and last elements of the array. After that, we re-apply Max-Heapify:

1:	7	6	5	4	2	1	0	3	8
2:	6	4	5	3	2	1	0	7	8
3:	5	4	1	3	2	0	6	7	8
4:	4	3	1	0	2	5	6	7	8
5:	3	2	1	0	4	5	6	7	8
6:	2	0	1	3	4	5	6	7	8
7:	1	0	2	3	4	5	6	7	8
8:	0	1	2	3	4	5	6	7	8

Finally sorted array using Heap-Sort:

0	1	2	3	4	5	6	7	8

2. A d-ary heap is similar to a binary heap, except non-leaf nodes have d children instead of 2 children (except the last non-leaf node, which is allowed to have fewer children). Adjust the array representation and the efficient implementations of Max-Heapify and Build-Max-Heap. **Write down** the generalized pseudocode of Max-Heapify and Build-Max-Heap for d-ary heaps for any d.

Pseudocode:

```
Max-Heapify(A, i, d):
    maximum = i
    for k = 1 to d:
        child = d * i + k
        if child < A.size() and A[child] > A[maximum]:
            maximum = child
    if maximum != i:
        swap(A[i], A[maximum])
        Max-Heapify(A, maximum, d)

Build-Max-Heap(A, d):
    for i = ((A.size() - 1) / d) to 0:
        Max-Heapify(A, i, d)
```

Then, perform a variation of Heap-Sort [CLRS, §6.4] using a 4-ary heap on the following input array:

2	3	0	4	7	1	5	8	6

Show the state of the array after each call to Max-Heapify from the Heap-Sort procedure (solution must contain 8 arrays and the pseudocode for generalized Max-Heapify and Build-Max-Heap).

Answer:
Initially, we need to apply Max-Heapify to our array. Here is our resulting array:

8	6	0	4	7	1	5	3	2

Next, for all other arrays, we apply an algorithm that will swap the first and last elements of the array. After that, we re-apply MAX-HEAPIFY:

1:	7	6	0	4	2	1	5	3	8
2:	6	5	0	4	2	1	3	7	8
3:	5	3	0	4	2	1	6	7	8
4:	4	3	0	1	2	5	6	7	8
5:	3	2	0	1	4	5	6	7	8
6:	2	1	0	3	4	5	6	7	8
7:	1	0	2	3	4	5	6	7	8
8:	0	1	2	3	4	5	6	7	8

Finally sorted array using HEAP-SORT:

0	1	2	3	4	5	6	7	8

3. Insert the $\langle \mathsf{key}, \mathsf{value}\rangle$ items into an empty B-tree [CLRS, §18] with minimum degree $t = 2$:

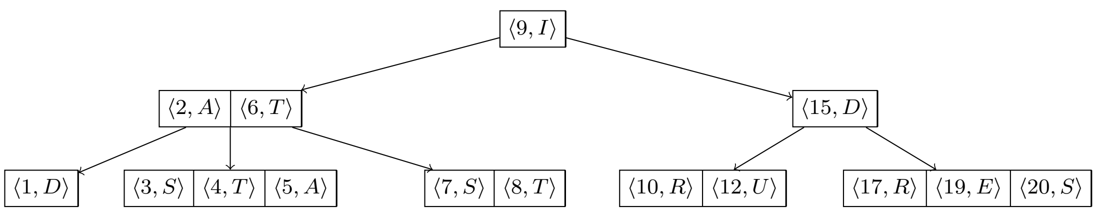
   (a) $\langle 32, I\rangle$, $\langle 17, I\rangle$, $\langle 9, X\rangle$, $\langle 21, C\rangle$, $\langle 11, E\rangle$

   (b) $\langle 2, A\rangle$, $\langle 28, E\rangle$, $\langle 36, G\rangle$, $\langle 3, E\rangle$, $\langle 4, T\rangle$

   (c) $\langle 18, B\rangle$, $\langle 13, E\rangle$, $\langle 6, T\rangle$, $\langle 7, R\rangle$, $\langle 37, I\rangle$

   (d) $\langle 26, L\rangle$, $\langle 33, N\rangle$, $\langle 20, U\rangle$, $\langle 24, I\rangle$, $\langle 30, D\rangle$

   Show the state of the tree after every 5 insertions. Depict each tree as a sequence of arrays for each layer. For example, consider this B-tree:



   The tree above must be depicted as follows:

(layer 1) [$\langle 9, I\rangle$]

(layer 2) [$\langle 2, A\rangle$ | $\langle 6, T\rangle$] [$\langle 15, D\rangle$]

(layer 3) [$\langle 1, D\rangle$] [$\langle 3, S\rangle$ | $\langle 4, T\rangle$ | $\langle 5, A\rangle$] [$\langle 7, S\rangle$ | $\langle 8, T\rangle$] [$\langle 10, R\rangle$ | $\langle 12, U\rangle$] [$\langle 17, R\rangle$ | $\langle 19, E\rangle$ | $\langle 20, S\rangle$]

   Answer:

   (a) Image of the tree after adding $\langle 32, I\rangle$, $\langle 17, I\rangle$, $\langle 9, X\rangle$, $\langle 21, C\rangle$, $\langle 11, E\rangle$:

(layer 1) [$\langle 17, I\rangle$]

(layer 2) [$\langle 9, X\rangle$ | $\langle 11, E\rangle$] [$\langle 21, C\rangle$ | $\langle 32, I\rangle$]

   (b) Image of the tree after adding $\langle 2, A\rangle$, $\langle 28, E\rangle$, $\langle 36, G\rangle$, $\langle 3, E\rangle$, $\langle 4, T\rangle$:

(layer 1) [$\langle 9, X\rangle$ | $\langle 17, I\rangle$ | $\langle 28, E\rangle$]

(layer 2) [$\langle 2, A\rangle$ | $\langle 3, E\rangle$ | $\langle 4, T\rangle$] [$\langle 11, E\rangle$] [$\langle 21, C\rangle$] [$\langle 32, I\rangle$ | $\langle 36, G\rangle$]

   (c) Image of the tree after adding $\langle 18, B\rangle$, $\langle 13, E\rangle$, $\langle 6, T\rangle$, $\langle 7, R\rangle$, $\langle 37, I\rangle$:

(layer 1) [$\langle 17, I\rangle$]

(layer 2) $\boxed{\langle 3, E\rangle \mid \langle 9, X\rangle}$ $\boxed{\langle 28, E\rangle}$

(layer 3) $\boxed{\langle 2, A\rangle}$ $\boxed{\langle 4, T\rangle \mid \langle 6, T\rangle \mid \langle 7, R\rangle}$ $\boxed{\langle 11, E\rangle \mid \langle 13, E\rangle}$ $\boxed{\langle 18, B\rangle \mid \langle 21, C\rangle}$ $\boxed{\langle 32, I\rangle \mid \langle 36, G\rangle \mid \langle 37, I\rangle}$

(d) Image of the tree after adding $\langle 26, L\rangle$, $\langle 33, N\rangle$, $\langle 20, U\rangle$, $\langle 24, I\rangle$, $\langle 30, D\rangle$:

(layer 1) $\boxed{\langle 17, I\rangle}$

(layer 2) $\boxed{\langle 3, E\rangle \mid \langle 9, X\rangle}$ $\boxed{\langle 21, C\rangle \mid \langle 28, E\rangle \mid \langle 36, G\rangle}$

(layer 3) $\boxed{\langle 2, A\rangle}$ $\boxed{\langle 4, T\rangle \mid \langle 6, T\rangle \mid \langle 7, R\rangle}$ $\boxed{\langle 11, E\rangle \mid \langle 13, E\rangle}$ $\boxed{\langle 18, B\rangle \mid \langle 20, U\rangle}$ $\boxed{\langle 24, I\rangle \mid \langle 26, L\rangle}$ $\boxed{\langle 30, D\rangle \mid \langle 32, I\rangle \mid \langle 33, N\rangle}$ $\boxed{\langle 37, I\rangle}$

References

[CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition.* MIT press.

[GTG] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser. *Data Structures and Algorithms in Java.* WILEY 2014.